

Your AI Pair Programmer

A 45-minute introduction for experienced developers — from completions to chat to customization.

Let's get started →

45 min · 19 slides



The Question This Talk Answers



"What can GitHub Copilot actually do for an experienced developer, and how do I get the most out of it from day one?"

Target: Experienced Developers · 45 Minutes

Table of Contents



The Landscape

Why now, how it works, the context model



Code Completions

Inline suggestions, ghost text, NES



Copilot Chat

Context variables, agents, workflows



CLI & Code Review

Terminal AI, automated PR review



Customization

Instructions, prompts, agents, MCP



Getting Started

Immediate actions, resources, next steps

The Landscape: AI-Assisted Development

Why Now?

60-70% of dev work is mechanical — boilerplate, API lookups, test scaffolding, docs

AI removes friction — not replacing developers, amplifying expertise

Integrated, not separate — lives in your editor, terminal, and PR workflow

The Full Surface Map



Completions — Inline suggestions as you type



Chat — Conversational AI with codebase context



CLI — Terminal command assistance

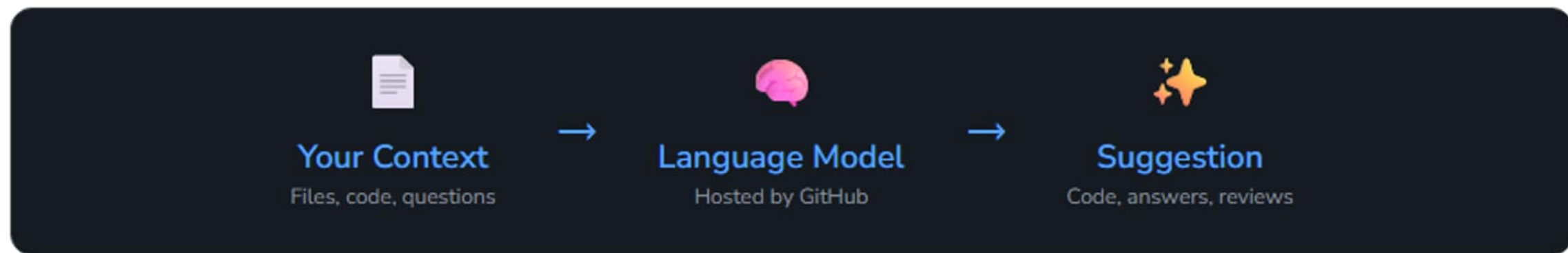


Code Review — Automated PR analysis



Customization — Team patterns & conventions

How Copilot Works



The Key Insight

The model is the same across all surfaces. What changes is the context each surface provides. More context = better suggestions.

Completions

Current file + open tabs

Chat

What you reference

CLI

Command + history

Code Review

PR diff + context



The Shift

The Mental Model

✗ "AI writes code for me"



✓ "AI amplifies my expertise when I give it the right context"

This shift applies to every surface you'll see today

Code Completions: Your Typing Accelerator

How It Works

Ghost text appears as you type — press `Tab` to accept, `Esc` to dismiss

Single lines to entire functions — Copilot infers intent from your code and comments

Next Edit Suggestions (NES) — predicts your next edit location after accepting

What Copilot Sees

- ◆ Current file content (before & after cursor)
- ◆ Open editor tabs (neighboring files)
- ◆ Language & framework from file extension
- ◆ Comments & function signatures as intent signals

Getting Better Completions

✓ Write intent before implementation

```
# Calculate the monthly subscription cost including tax  
# and any active discounts. Return final amount rounded to 2 decimals.  
def calculate_monthly_cost(subscription, tax_rate, discounts):
```

Copilot now knows the full intent — not just "write a function"

✗ Vague naming

```
function process(data) { }
```

✓ Specific naming

```
function validateUserRegistration(  
  formData: RegistrationForm  
) { }
```

💡 Pro tip: Open relevant files in editor tabs — Copilot uses them as context for completions

Completions: Patterns That Work

High-Value Patterns

Boilerplate generation — CRUD endpoints, model definitions, config files

Pattern continuation — Write one example, Copilot follows for the rest

Test scaffolding — Describe the test in a comment, get the implementation

Documentation — JSDoc, docstrings, README sections from code context

Know the Limits

Always review — suggestions may look right but have subtle bugs

Novel algorithms — Copilot is better at applying known patterns than inventing new ones

Business logic — complex domain rules need explicit specification

Completions keep you in **flow state** — for deeper tasks, switch to **Chat**

Copilot Chat: Your Codebase Expert



Chat Panel

Side panel for extended conversations and complex questions



Inline Chat

`Ctrl+I` — Quick edits right in your code



Quick Chat

`Ctrl+Shift+I` — Fast questions, no context switch



Key Difference from Completions

Completions are reactive (suggestions as you type). Chat is proactive (you ask questions, explore, and direct). Use completions for flow, Chat for exploration.

Chat Context Variables: The Power Feature

Context variables tell Chat exactly what to look at — the difference between generic and codebase-specific answers.

#file — Reference a specific file

"Explain #file:auth.ts"

@workspace — Search entire workspace

"@workspace how is auth implemented?"

#codebase — Semantic search

"#codebase find error handling patterns"

#selection — Selected code

Select code → "Refactor #selection"

@terminal — Terminal context

"@terminal why did my last command fail?"

@vscode — Editor features

"@vscode how do I enable word wrap?"

💡 The more specific your context, the better the response. "@workspace how does auth work" is good. "Explain the JWT validation in #file:middleware/auth.ts" is much better.

Chat in Practice

Debugging with Context

```
I'm getting a NullReferenceException in #file:OrderService.cs at line 47.  
@workspace show me how other services handle null order lookups.
```

Architecture Exploration

```
@workspace explain the authentication flow from login to token refresh.  
Include which files are involved.
```

Code Generation with Constraints

```
Generate a REST endpoint for user profiles following the patterns  
in #file:UserController.cs. Use the same error handling approach.
```

Chat: Agent Mode

What Is Agent Mode?

Autonomous multi-step execution — Copilot plans, implements, and iterates

Multi-file edits — creates, modifies, and deletes files across your project

Terminal commands — runs build, test, and lint automatically

Self-correction — detects errors and fixes them without manual intervention

When to Use It



Multi-file refactoring



New feature scaffolding



Test generation across modules



Bug fixing with iteration



Always review changes before accepting

Agent mode = Chat + code editing + terminal in one autonomous loop

Copilot in the CLI

Terminal AI Assistance

Interactive conversations and programmatic execution — right in your terminal.

```
# Interactive mode — start a session
$ copilot
> "Explain what this command does:
  find . -name '*.log' -mtime +30 -delete"
```

```
# Ask for help with a task
> "Compress all PNGs in this dir recursively"
```

```
# Programmatic mode — single command
copilot -p "Why did the build fail?" \
  --allow-tool 'shell(command)'
```

Why It Matters

Plan Mode — press Shift+Tab to plan before coding

No context switching — stay in your terminal, ask naturally

Two modes — interactive for exploration, `-p` for scripts/CI

Install:

```
gh copilot (auto-installs) or npm i -g @github/copilot
```

Copilot Code Review

Automated PR Analysis

Copilot reviews your pull requests and provides specific, actionable feedback — inline comments just like a human reviewer.

What It Catches

-  Logic errors & edge cases
-  Security vulnerabilities (SQL injection, XSS)
-  Performance issues (N+1 queries)
-  Missing error handling
-  Codebase inconsistencies

How to Use

Enable in repository settings

Or request review from `@copilot`

Comments appear inline on the diff

Each comment includes a suggested fix

 Code Review handles mechanical checks so human reviewers focus on architecture and design

Customization: Teaching Copilot Your Patterns

Out of the box, Copilot knows general programming. With customization, it knows **your team's** programming.



Instructions

`.github/copilot-instructions.md` — Team conventions applied to every response



Custom Prompts

`.github/prompts/*.prompt.md` — Reusable, shareable prompt templates



Agent Skills

`.github/skills/*/SKILL.md` — Domain knowledge loaded when relevant



MCP Servers

`.vscode/mcp.json` — Connect to databases, APIs, external tools



Custom Agents

`.github/agents/*.agent.md` — Role-based AI specialists

How Customization Compounds

No Customization

"Create a user endpoint"

→ Generic REST boilerplate

With Instructions

"Create a user endpoint"

→ Follows your error handling, naming, auth patterns

Full Stack

"Create a user endpoint"

→ Follows patterns + uses prompt template + queries DB schema

Quick Start: Create copilot-instructions.md

```
# Project Instructions
- Use TypeScript with strict mode
- Follow REST conventions: plural nouns, proper HTTP methods
- All endpoints require authentication via JWT
- Use Zod for request validation
- Return consistent error format: { error: string, code: number }
```

This single file transforms every Copilot interaction from generic to team-specific — 5 minutes to create.

Which Surface When?



Completions

Writing code in flow

Auto context

Instant



Chat

Exploration & questions

Explicit context

Seconds



CLI

Terminal tasks

Command context

Seconds



Code Review

PR quality

PR diff

Minutes

The golden rule: Use completions to stay in flow. Switch to Chat when you need to think. Use CLI in the terminal. Let Code Review catch what you missed.



Mental Model Shift

✓ Move Toward

Context-First Thinking: Open relevant files, use #file refs → architecture-aware suggestions

Conversational Iteration: Use Chat for dialogue → complex problems solved through refinement

Multi-Surface Workflow: Each surface for its strength → completions for flow, Chat for exploration

Teach Your Patterns: copilot-instructions.md → every team member gets consistent suggestions

● Move Away From / Against

Tab-Only Usage: Ignoring Chat, CLI, Review → missing 80% of Copilot's value

Zero-Context Prompting: No file refs → generic responses

Manual Everything: Hand-writing boilerplate, tests, docs → time on tasks AI handles well

Blind Acceptance: No review of suggestions → AI bugs are still bugs

Replacing Understanding: Code you can't explain → unmaintainable debt



Getting Started Today

🕒 15 Minutes

- Try completions with awareness — write a comment, then let Copilot implement
- Open Chat panel: `@workspace explain this project's architecture`
- Try inline Chat: `Ctrl+I` on a function: "Add error handling"

🕒 1 Hour

- Create `.github/copilot-instructions.md` with 5-10 team conventions
- Install CLI: `gh copilot` (auto-installs) or `npm i -g @github/copilot`
- Request Copilot Code Review on your next PR

🕒 2-4 Hours

- Create 2-3 `.prompt.md` files for common tasks
- Explore Agent mode for a multi-file change
- Set up an MCP server for your database or API



Resources & Next Steps

Official Documentation

 [GitHub Copilot Docs](#) — Complete reference

 [VS Code Copilot Guide](#) — Editor integration

 [Copilot CLI Docs](#) — Terminal usage

Go Deeper

 [Copilot Chat](#) — Context mastery deep dive

 [Copilot CLI](#) — Terminal workflow optimization

 [Code Review](#) — PR automation patterns

 [Full Workshop](#) — 8-9 hours hands-on customization

Remember: **More context = better suggestions.** That's the thread connecting everything you've seen today.



 Thank You

Start with context. The rest follows.

github.com/copilot